



JAVA对数据库的基本操作

华为云 + 智能, 见未来

www.huaweicloud.com



前言

- Java基于JDBC可以实现对数据库的操控与处理。本章将在java语言下结合相应代码实践，完成环境端到端的搭建与配置，利用程序完成对数据库的基本操作与数据处理。



课程目标

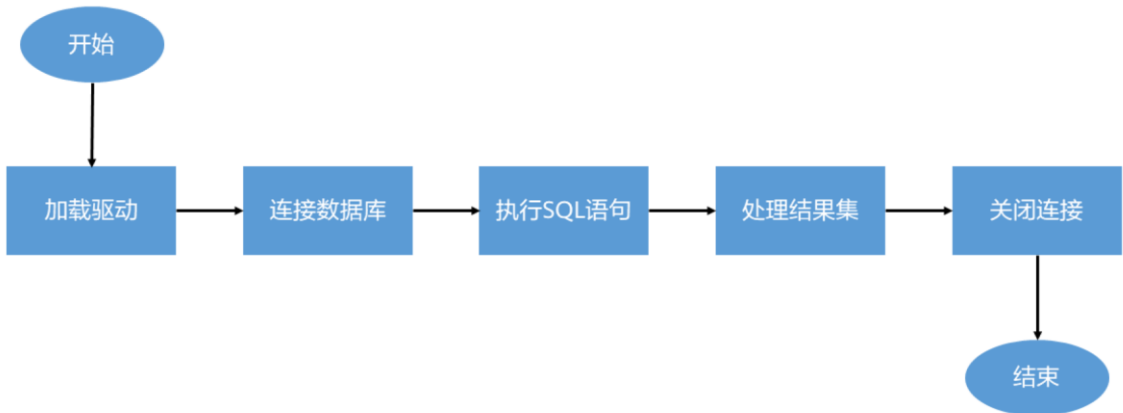
- 学完本课程后，您将能够：
 - 掌握使用JAVA程序对数据库执行基本操作。

目录

1. 开发流程概述

2. 基本操作

JDBC开发应用程序的流程



- 本章内容将围绕着这个流程展开。

JDBC包

- 从官网获取JDBC包，GaussDB(for openGauss)提供的JDBC驱动包。包名为GaussDB-Kernel-VxxxRxxxCxx-操作系统版本号-64bit-Jdbc.tar.gz。
- 解压后有两个JDBC的驱动jar包：
 - gsjdbc4.jar：与PostgreSQL保持兼容的驱动包，其中类名、类结构与PostgreSQL驱动完全一致，曾经运行于PostgreSQL的应用程序可以直接移植到当前系统使用。
 - gsjdbc200.jar：如果同一JVM进程内需要同时访问PostgreSQL及GaussDB(for openGauss)时，需使用此驱动包。
- 在创建数据库连接之前，需要加载数据库驱动类“org.postgresql.Driver”。

- 参考链接为：https://dbs-download.obs.cn-north-1.myhuaweicloud.com/GaussDB/1642684986086/GaussDB_opengauss_client_tools.zip
- 不推荐使用gsjdbc200.jar安装包

目录

1. 开发流程概述

2. 基本操作

加载驱动

- 在创建数据库连接之前，需要先加载数据库驱动程序。
- 加载驱动有两种方法：
 - 在代码中创建连接之前任意位置隐含装载：

```
Class.forName("org.postgresql.Driver");
```

- 在JVM启动时参数传递：

```
java -Djdbc.drivers=org.postgresql.Driver jdbctest
```


配置环境

- 客户端需配置JDK1.8版本，JDK是跨平台的，支持Windows/Linux等多种平台，下面以Linux为例，介绍JDK配置流程：
 - 获取JDBC驱动和JDK软件包，并将软件包上传至客户端处。
 - 将安装路径添加至java_home/classpath/path环境变量中。
 - 输入“java -version”，查看JDK版本，确认为JDK1.8版本。
 - 输入“javac -version”，查看javac是否正常运行。

连接数据库

- JDBC提供了三个方法，用于创建数据库连接，基础实践中使用以下方法连接数据库。

```
DriverManager.getConnection(String url);
```

- URL: `jdbc:postgresql://host:port/database?param1=value1¶m2=value2`
 - 如果使用gsjdbc200.jar时，需要将“jdbc:postgresql”修改为“jdbc:gaussdb”
 - host为数据库服务器名称或IP地址，此处为CN的地址
 - port为数据库服务器端口，缺省情况下会连接到5431端口
 - param为参数名称，即数据库连接属性。参数可以配置在URL中，以“?”开始配置，以“=”给参数赋值，以“&”作为不同参数的间隔。
 - 连接时需配置connectTimeout和socketTimeout，推荐配置为2（如果未配置，默认为0，即不会超时）。在DN与客户端出现网络故障时，客户端一直未收到DN侧ACK确认报文，会启动超时重传机制，不断的进行重传。当超时时间达到系统默认的600s后才会报超时错误，这也会导致RTO时间很高。
 - 例如: `jdbc:postgresql://192.168.0.16:8000/postgres?connectTimeout=2&socketTimeout=2`

执行SQL

- 程序中，一般使用Statement接口和PreparedStatement接口执行SQL语句。
- 对于较为简单的create table建表语句与insert插入单条记录语句，使用了Statement接口，应用程序通过该接口执行SQL语句来操作数据库的数据，执行不用传递参数的语句；
- PreparedStatement接口则主要执行较为复杂的语句，预编译语句是只编译和优化一次，然后通过设置不同的参数值多次使用。实验中的update更新、delete删除和批量插入操作，都使用了PreparedStatement接口。

执行SQL – 批处理

- 执行SQL时，往往会进行批量处理，如果采用Statement的方式执行，执行效率较低，因此对于批处理，采用PreparedStatement的方式。
- 调用Connection的prepareStatement方法创建预编译语句对象。

```
Connection conn = DriverManager.getConnection("url","user","password");  
PreparedStatement pstmt = conn.prepareStatement("INSERT INTO test VALUES (?)");
```

- 针对每条数据都要调用setShort设置参数，以及调用addBatch确认该条设置完毕。

```
pstmt.setShort(1, (short)2);  
pstmt.addBatch();
```

- 调用PreparedStatement的executeBatch方法执行批处理。

```
int[] rowcount = pstmt.executeBatch();
```

- 调用PreparedStatement的close方法关闭预编译语句对象。

```
pstmt.close();
```

- 批处理就是对某对象进行批量的处理，如批量处理SQL语句，而不是单次执行一条。

结果集

- 不同类型的结果集有各自的应用场景，应用程序需要根据实际情况选择相应的结果集类型。在执行SQL语句过程中，都需要先创建相应的语句对象，而部分创建语句对象的方法提供了设置结果集类型的功能。
- 创建一个Statement对象，该对象将生成具有给定类型和并发性的ResultSet对象。
 - `createStatement(int resultSetType, int resultSetConcurrency);`
- 创建一个PreparedStatement对象，该对象将生成具有给定类型和并发性的ResultSet对象。
 - `prepareStatement(String sql, int resultSetType, int resultSetConcurrency);`

使用ResultSet接口方法处理结果集

- ResultSet对象具有指向其当前数据行的指针。最初，指针被置于第一行之前。next方法将指针移动到下一行；next方法在ResultSet对象没有下一行时，会返回false，所以可以在while循环中使用它来迭代结果集。但对于可滚动的结果集，JDBC驱动程序提供更多的定位方法，使ResultSet指向特定的行。
- 示例：

```
//此处省略连接数据库的代码...
Statement stmt =conn.createStatement();
ResultSet rs =stmt.executeQuery("select * from test");
while(rs.next()){
    for (int i = 0; i < metaData.getColumnCount(); i++) {
        System.out.print(rs.getString(i + 1) + "\t");
    }
}
```

关闭连接

- 在使用数据库连接完成相应的数据操作后，需要关闭数据库连接。
- 很多的数据库类如Connection、Statement和ResultSet都有close()方法，在使用完对象后应把它们关闭。需要注意的是，Connection的关闭将间接关闭所有与它关联的Statement，Statement的关闭间接关闭了ResultSet。

总结

- 本章在java语言下，完成环境端到端的搭建与配置，实现了程序对数据库的基本操作与数据处理，应用的数据持久化便有了基石。在实际业务场景中，基于此便可抽象构建出应用的数据接入层，便于应用业务层对应数据层的复用、扩展与解耦。

思考题

1. （判断题）当使用批量SQL插入时，推荐采用Statement的方式执行。（ ）
 - A. True
 - B. False
2. （单选题）配置URL时，使用以下哪种字符作为不同参数的间隔？（ ）
 - A. ?
 - B. =
 - C. &
 - D. \$

- B。建议使用PreparedStatement方式
- C。使用&字符作为不同参数间隔。

缩略语

- JDBC, Java Database Connectivity, Java数据库连接。
- API, Application Programming Interface, 应用程序接口。
- URL, Uniform Resource Locator, 统一资源定位系统。



感谢

版权所有©2022，华为技术有限公司，保留所有权利。

本资料是华为的保密信息，所有内容仅供华为授权的培训客户内部使用，禁止用于任何其他用途。未经许可，任何人不得对本资料进行复制、修改、改编，也不得将本资料或其任何部分或基于本资料的衍生作品提供给他人。

华为云 + 智能，见未来

www.huaweicloud.com